

能力契约驱动的 企业级 Agent 执行范式

Agent 主导的 Dynamic Workflow

演讲嘉宾：魏粲实

DataFun · Agentic AI Summit

目录

CONTENT

从任务状态，到能力空间，再到执行轨迹

01 问题重构

企业级任务，到底有什么不同？

02 常见范式

常见做法，他们的边界在哪里

03 行业信号

执行为什么开始走出 Agent？

04 执行架构

能力怎样交付，Agent 怎样执行，系统怎么承接

05 能力驱动的企业级 Agent

对话驱动到能力驱动

01

问题重构

企业级 Agent 到底在执行什么？

企业级任务不是“生成答案”，而是“推进状态”

同样叫 Agent，企业级任务多了三类责任：证据、验证、状态提交

CONSUMER AGENT



ENTERPRISE AGENT



企业级任务不只是生成答案，而是推进一个可审计的业务状态向前走

只要有状态，就会有边界、恢复和责任。

只要任务会改变系统状态，workflow 语义就会回来。

一次投诉升级，原本要完成什么？

一次投诉升级：从异常的定位，到沉淀策略

业务目标

定位异常原因

形成处理建议

沉淀优化策略

更新业务状态

一次处理，反哺回系统，
形成下一次判断的依据

稳定 SOP



工单进来，证据支撑归因
必要时人审，结果进入归档
归档后，再反哺标签、规则和策略

系统要求

证据可追踪

责任可解释

风险可判断

策略可复用

结果可归档

SOP 能稳定主干，但是在真实场景中，很难每次都会沿着这条主干顺利的向前走。

同一条业务闭环上，下一步会被证据改写

输入证据包

- ticket 投诉对象与状态
- SLA 服务承诺边界
- alert timeline 异常发生顺序
- chat logs 上下文证据

很难保证这些证据
始终完整、一致

稳定 SOP



同一条 SOP 在执行过程中，可能出现回退、跳转，甚至循环

SOP 定义的是稳定的业务主干，真实执行路径由当时的证据和状态决定。

状态账本

每次路径变化，都要回答两个问题

为什么改？

evidence_ref

decision_reason

改完能不能提交？

risk_level

commit_policy

提交前验证

01 路径怎么调整？

路径能不能根据现场情况调整

- 证据缺失时，能不能先补证
- 结论冲突时，能不能回退重判
- 目标满足时，能不能提前结束

路径可以调整，但系统接不住，
最后就是一个很自由、却没法负责的 Agent。

02 结果怎么落地？

调整以后产生的结果，系统能不能接得住

- 证据、权限、风险能不能解释
- 验证失败时，能不能恢复
- 结果通过后，能不能提交状态

系统边界很稳，但路径完全不能改，
又会回到一张越来越复杂的流程图。

Agent设计的差异，就在于这两个问题回答到什么程度

02

常见范式

动态判断被放在哪里？

判断留在设计阶段 vs 判断进入执行过程

Workflow / Chain 并不等于固定串行流程

01 Prompt Chaining

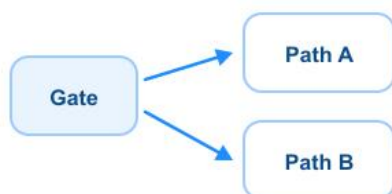
顺序编排



一个任务拆成连续的处理步骤

02 Routing

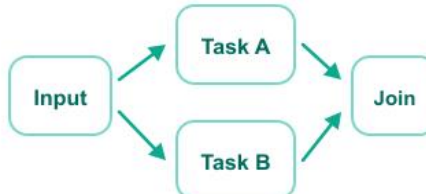
条件路由



根据用户意图选择不同的处理分支

03 Parallelization

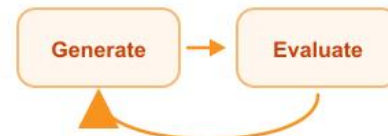
并行与汇合



任务并行展开再汇合

04 Evaluator-Optimizer

评价与迭代



多轮评价和修正

LLM 可以进入任务节点，也可以承担分类、评价和局部任务拆分

工程价值

控制流、状态传递、失败处理显式可见

可测试 · 可监控 · 可运营

设计边界

运行时可以动态，编排协议不能临时改写

跨阶段关系、状态语义与完成条件仍由 Workflow 定义

Workflow 不缺少动态行为；它把动态行为组织在预先定义的控制协议里。

Agent + Skill: Agent 负责执行, Skill 提供专业方法

Agent + Skill: Agent 负责执行, Skill 提供专业方法



Skill 让通用 Agent 成为领域专家，但不是一套执行协议。

写进 Skill 的 workflow，首先是一套给模型理解和遵循的方法论，并不是一套由执行引擎强制调度的控制流

Agent in Workflow: Agent 成为 Workflow 中的执行节点

Act 02

Agent in Workflow: Agent 成为 Workflow 中的执行节点



Agent 获得的是节点内的自主权，Workflow 保留的是链路级控制权。

三种形态的差别，在于控制权如何分配

Act 02

三种形态的差别，在于控制权如何分配

差别不在于谁更动态，而在于控制权放在哪里。

Workflow / Chain

Workflow 控制整条业务链路



控制范围：整条链路

Agent + Skill

Agent 控制一次任务执行



按需加载 Skill

控制范围：当前任务

Agent in Workflow

Workflow 控制链路，
Agent 控制节点内部



控制范围：链路 + 节点内部

动态判断不是有或没有，而是被放在了不同的执行层级。

把动态判断权交给 Agent 的同时，必须明确业务控制权由谁持有。

路径可以动态，业务约束必须保持稳定

Agent 的自由度不能带着判断依据和完成标准一起漂移。



工程问题：怎么把动态执行和稳定的业务约束拆开？

怎么把动态执行和稳定的业务约束拆开？

03

行业信号

执行过程正在从对话里外置出来。

典型场景：代码库级审计 · 迁移 · 重构

一个 Agent 会话无法协调



计划、分支和中间结果都留在对话上下文



subagents

负责局部判断

collect

汇总中间结果

reviewer

交叉校验输出

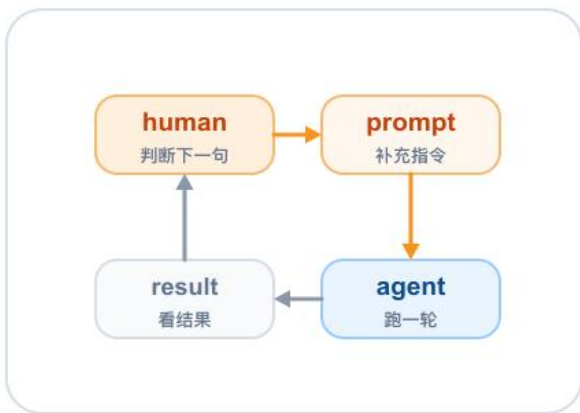
Agent 负责局部判断，script 负责整体编排。

计划从对话上下文，迁移到可执行、可重跑的外部编排。

Loop Engineering: 人不再接下一句, 而是设计下一轮

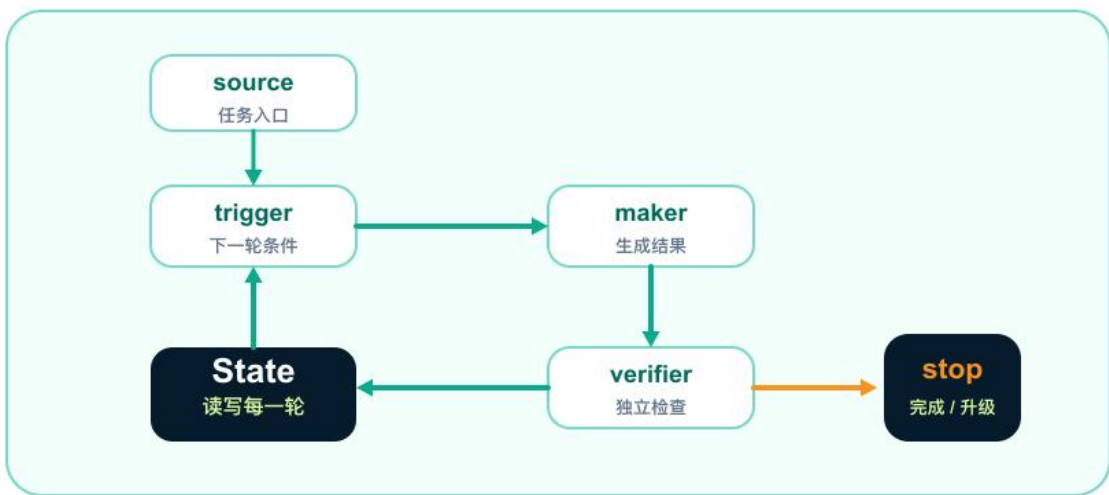
Claude Code: 从产品形态上将执行过程外置 → Loop Engineering: 从人的工作方式讲同一件事情

过去: 人自己就是 loop



看结果 → 补 prompt → 再跑一轮

现在: 设计一个能持续驱动 Agent 的系统循环



人的工作就从“接下一句”, 变成设计这个循环本身

人不再逐轮 prompt Agent, 而是设计一个持续驱动 Agent 的系统循环。

外部结构承接了控制，但具体路径没有提前锁死

Act 03

执行编排从对话里拿出来，只是第一步

PATH EMERGES AT RUNTIME

外部结构承接了控制，但具体路径没有提前锁死

外部结构持续保存当前状态，也负责驱动结束后结果检查



Dynamic 的含义：控制结构保持稳定，具体路径由每一轮的任务状态和校验结果逐步形成

下一步由当前任务状态和校验结果决定。

04

执行架构

当路径在运行中展开，结构由系统承接，交付的重点也要发生变化
设计时交付能力契约，运行时生成执行轨迹。

OpenSpec Signal: 从 Phase Workflow 到 Action-Oriented Workflow

OpenSpec: 面向 AI 编码的轻量级规范层

用可追踪、可修订的 artifacts, 组织一次软件变更

传统工作流: phase-locked

计划、实现、完成被当成阶段门



发现偏差时: Can't go back

Actions, Not Phases

不是不要过程, 而是不要把工作锁死在一个个阶段门里

artifacts are the live plan



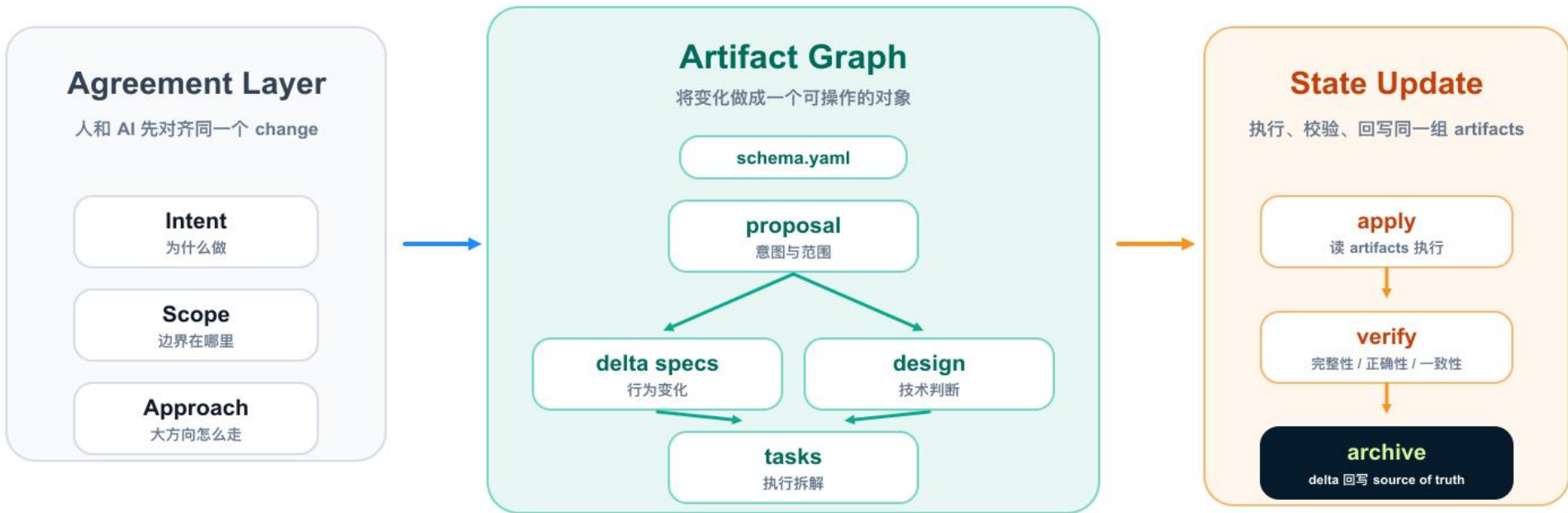
update as you learn

核心哲学: 不是回到某个阶段, 而是随时更新当前 artifacts。

动态不是取消结构, 而是让 action 可以在执行中修正前置判断。

Artifact Graph: 承接变化的可验证上下文

Agreement Layer → Artifact Graph → Source of Truth



执行过程中无论在哪一步发现问题，都可以回到对应的 artifact 进行修正
稳定性来自每一次的回溯都有对象可改、有依据可查、有结果可验

放到企业级 Agent 中，与 artifact graph 对应的，也就是我们的业务状态、证据、调用记录和提交结果。

Skill Set Package: 设计时交付的能力边界

设计时交付物



RUNTIME 加载



按照运行职责拆分



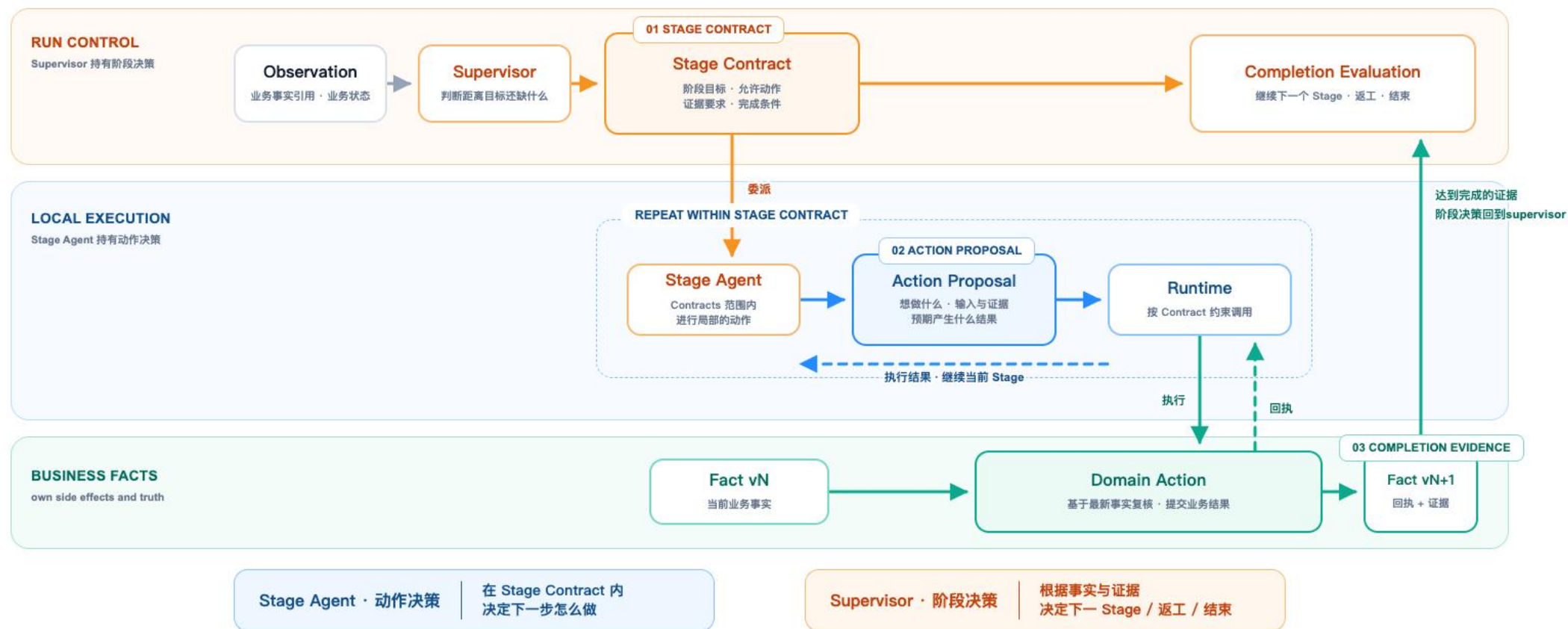
设计时交付的不是一条执行路径，而是一套可以在运行时生成契约的能力边界。

Agent Run Architecture: 动作决策与阶段决策分离

Act 04

ONE RUN · TWO DECISIONS · ONE STAGE CONTRACT

REFERENCE ARCHITECTURE



Stage Agent 决定局部动作; Supervisor 根据事实与证据决定下一阶段。

Stage Contract: 约束委派和完成判断

同一份 Contract, 在 Stage 开始和结束时各使用一次

ONE CONTRACT · TWO CONTROL POINTS



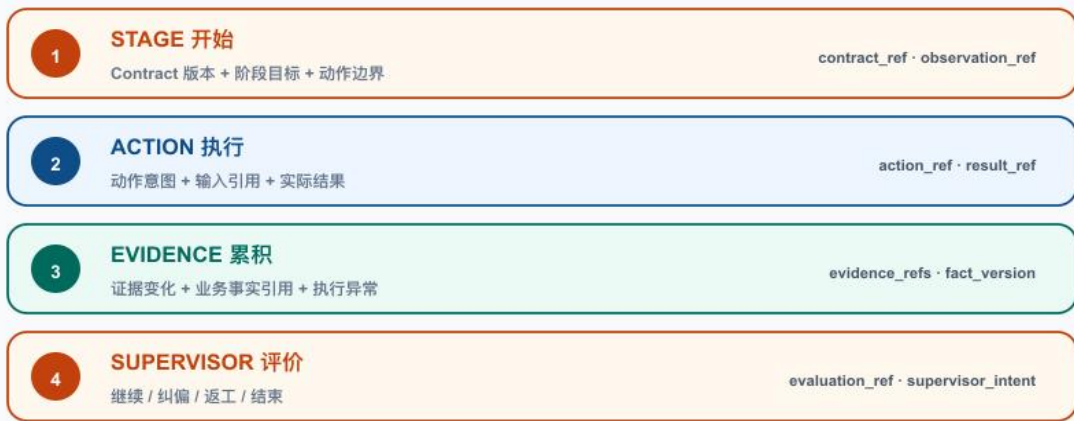
同一份 Contract, 在 Stage 开始和结束时各使用一次。

Execution Ledger: 让执行状态可恢复、可审计

执行路径可以变化，系统必须知道现在走到哪里，以及为什么

Execution Ledger

按顺序追加执行状态与证据引用



注意边界：Ledger 记录执行状态和引用，权威业务事实仍由 Domain System 持有。

这些记录直接参与后续运行

中断恢复

从最近一次确定的执行状态继续

路径重建

重建动作、证据与判断之间的因果关系

评价失败后的问题诊断

Supervisor 回看轨迹，定位从哪一步开始偏离

Ledger 可以恢复执行进度；已经发生的业务结果，需要通过业务补偿处理。

执行路径可以变化，系统必须知道现在走到哪里，以及为什么。

Supervisor Evaluation: 沿执行轨迹定位并纠正偏差

Act 04

评价失败只是结果，真正的纠偏要沿着执行轨迹定位偏离点

EVALUATION-DRIVEN CORRECTION

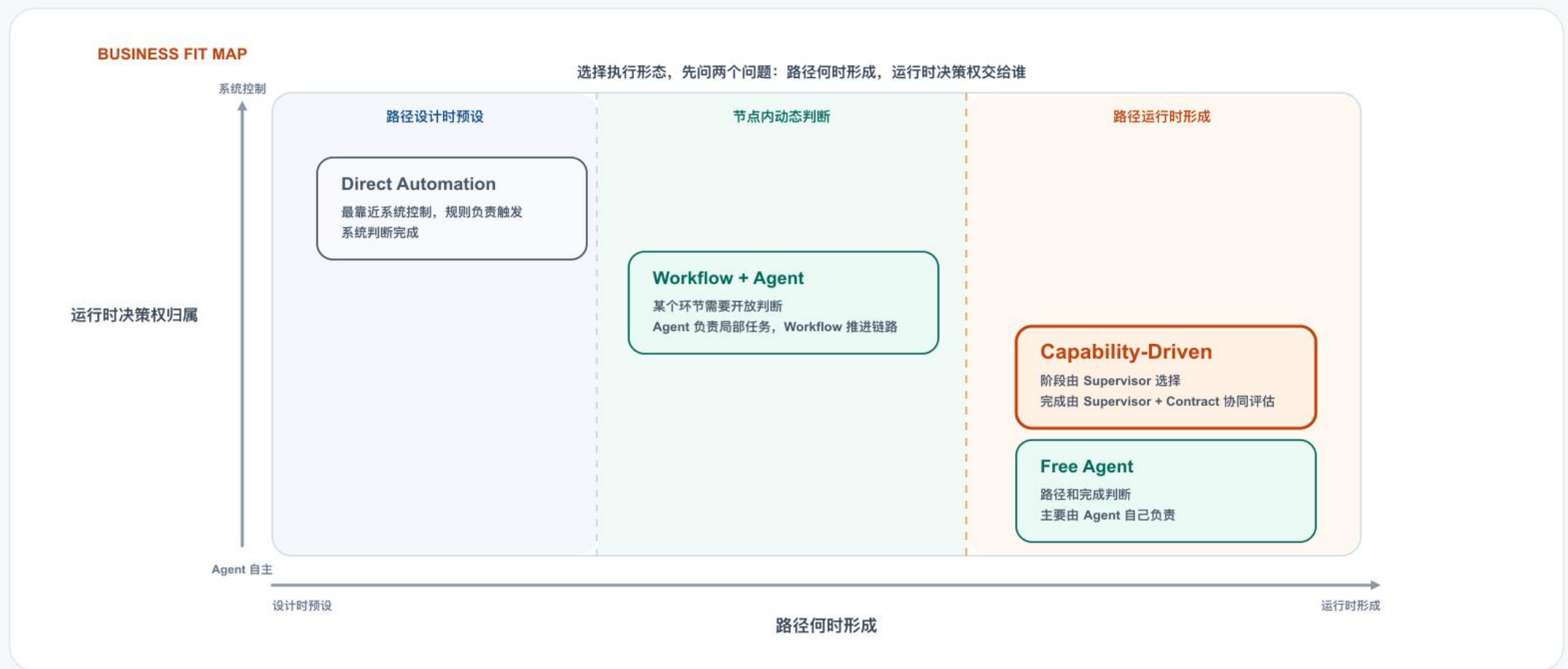


自进化从在线纠偏开始：执行轨迹成为下一轮 Supervisor 意图的依据。

评价不通过时，Supervisor 沿 Ledger 定位偏差，并向当前 Stage 传递纠偏意图。

Architecture Fit: 不是所有业务都值得动态化

Act 04



这四种形态不是能力强弱排序，而是系统控制和 Agent 自主之间不同的分工。

05

能力驱动的企业级 Agent

一项业务如何跨越多次执行保持连续

持续运行的主体，从 Agent Session 转向 Business Capability。

Agent Session 会结束, Business Capability 持有业务连续性

真正需要持续运行的, 不是 Agent Session, 而是业务本身

TRANSIENT SESSION · PERSISTENT CAPABILITY



Session 可以结束; 下一次执行仍从同一 Business State 继续。
Agent、系统、人工和外部事件围绕 Business State 接力, 形成 Dynamic Workflow。

持续保存的是目标、约束、完成条件和当前状态。

Capability 驱动的 Dynamic Workflow

CAPABILITY

持续存在的业务运行空间

BUSINESS STATE

业务进展与能力激活依据

AGENT HARNESS

开放判断的执行能力

DYNAMIC WORKFLOW

Business State 的实际迁移轨迹